

ISE 2

Name :- Parth Uttam Patil

Class :- MCA 1 Semester :- II

Subject :- Modern Full-Stack Development [MERN Authentication Project]

Roll No. :- 91

In this mini project, create a simple authentication system. The system allows new users to create an account using a Sign Up form and existing users to login using a Sign In form. User data will be stored in MongoDB database. Backend APIs will handle authentication logic and frontend React components will manage UI.

SignUp.js A simple Sign up form to register a User.

```
import React, { useState } from 'react';
```

```
const SignUp = () => {
```

```
  const [formData, setFormData] = useState({
```

```
    firstName: "",
```

```
    lastName: "",
```

```
    email: "",
```

```
    password: ""
```

```
  });
```

```
  const handleChange = (e) => {
```

```
    setFormData({ ...formData, [e.target.name]:
```

```
    e.target.value }));
```

```
  };
```

```
  const handleSubmit = (e) => {
```

```
    e.preventDefault();
```

```
    console.log("Sign Up Data:", formData);
```

```
// You will connect this to your Express backend  
later
```

```
};
```

```
return (
```

```
<div className="container">
```

```
<style>{`
```

```
.container {
```

```
  display: flex;
```

```
  justify-content: center;
```

```
  align-items: center;
```

```
  min-height: 100vh;
```

```
  /* Light, bright, and modern gradient */
```

```
  background: linear-gradient(135deg, #e0c3fc  
0%, #8ec5fc 100%);
```

```
  font-family: 'Inter', 'Segoe UI', Tahoma,  
Geneva, Verdana, sans-serif;
```

```
}
```

```
.form-card {
```

```
  /* Soft, slightly translucent white card */
```

```
  background: rgba(255, 255, 255, 0.95);
```

```
  padding: 45px 40px;
```

```
  border-radius: 20px;
```

```
  box-shadow: 0 20px 40px rgba(0, 0, 0, 0.08);
```

```
  width: 100%;
```

```
max-width: 420px;

border: 1px solid rgba(255, 255, 255, 0.5);

backdrop-filter: blur(10px);

}

.form-title {

text-align: center;

color: #2d3748;

margin-bottom: 28px;

font-size: 26px;

font-weight: 700;

letter-spacing: -0.5px;

}

.input-group {

margin-bottom: 20px;

}

.input-group label {

display: block;

margin-bottom: 8px;

color: #4a5568;

font-size: 14px;

font-weight: 500;

}

.input-group input {

width: 100%;
```

```
padding: 14px;

background-color: #f7fafc;

border: 2px solid transparent;

border-radius: 12px;

font-size: 15px;

transition: all 0.3s ease;

box-sizing: border-box;

color: #2d3748;

}

.input-group input:focus {

border-color: #8ec5fc;

background-color: #ffffff;

outline: none;

box-shadow: 0 0 4px rgba(142, 197, 252,
0.2);

}

.input-group input::placeholder {

color: #a0aec0;

}

.submit-btn {

width: 100%;

padding: 15px;

background: linear-gradient(135deg, #667eea
0%, #764ba2 100%);

color: white;
```

```
border: none;

border-radius: 12px;

font-size: 16px;

font-weight: 600;

cursor: pointer;

transition: transform 0.2s ease, box-shadow
0.2s ease;

margin-top: 10px;

}

.submit-btn:hover {

transform: translateY(-2px);

box-shadow: 0 8px 20px rgba(118, 75, 162,
0.3);

}

.submit-btn:active {

transform: translateY(0);

}

`}</style>
```

```
<div className="form-card">

  <h2 className="form-title">Create an
Account</h2>

  <form onSubmit={handleSubmit}>

    <div className="input-group">

      <label>First Name</label>
```

```
<input  
  
  type="text"  
  
  name="firstName"  
  
  placeholder="e.g. Swapnil"  
  
  value={formData.firstName}  
  
  onChange={handleChange}  
  
  required  
  
/>
```

```
</div>
```

```
<div className="input-group">
```

```
  <label>Last Name</label>
```

```
  <input
```

```
    type="text"
```

```
    name="lastName"
```

```
    placeholder="e.g. Malkar"
```

```
    value={formData.lastName}
```

```
    onChange={handleChange}
```

```
    required
```

```
  />
```

```
</div>
```

```
<div className="input-group">
```

```
  <label>Email Address</label>
```

```
  <input
```

```
    type="email"
```

```
        name="email"

        placeholder="you@example.com"

        value={formData.email}

        onChange={handleChange}

        required

    />

</div>

<div className="input-group">

    <label>Password</label>

    <input

        type="password"

        name="password"

        placeholder="Create a strong password"

        value={formData.password}

        onChange={handleChange}

        required

    />

</div>

<button type="submit" className="submit-
btn">Sign Up</button>

</form>

</div>

</div>

);
```

```
};
```

```
export default SignUp;
```

[Sign In](#) [Sign Up](#)

Create an Account

First Name

Parth

Last Name

Patil

Email Address

parth@gmail.com

Password

.....| 

Sign Up

SignIn.js Sign In page for the user.

```
import React, { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';

const SignIn = () => {
  const navigate = useNavigate();
  const [credentials, setCredentials] = useState({
    email: "",
    password: ""
  });

  const [message, setMessage] = useState({ text: "", isError: false });

  const handleChange = (e) => {
    setCredentials({ ...credentials, [e.target.name]: e.target.value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();

    if (credentials.email && credentials.password) {
      setMessage({ text: "Login Successful! Redirecting...", isError: false });

      setTimeout(() => {
        navigate('/welcome');
      }, 1500);
    } else {
      setMessage({ text: "Invalid Email or Password", isError: true });
    }
  };

  return (
    <div className="container">
      <style>{`
        .container {
          display: flex;
          justify-content: center;
          align-items: center;
          min-height: 100vh;
          /* Matches the SignUp gradient */
          background: linear-gradient(135deg, #e0c3fc 0%, #8ec5fc 100%);
          font-family: 'Inter', 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
        }
        .form-card {
          background: rgba(255, 255, 255, 0.95);
          padding: 45px 40px;
          border-radius: 20px;
          box-shadow: 0 20px 40px rgba(0, 0, 0, 0.08);
          width: 100%;
          max-width: 420px;
          border: 1px solid rgba(255, 255, 255, 0.5);
          backdrop-filter: blur(10px);
        }
        .form-title {
```

```

    text-align: center;
    color: #2d3748;
    margin-bottom: 8px;
    font-size: 26px;
    font-weight: 700;
    letter-spacing: -0.5px;
  }
  .form-subtitle {
    text-align: center;
    color: #718096;
    margin-bottom: 28px;
    font-size: 15px;
  }
  .message {
    text-align: center;
    padding: 12px;
    margin-bottom: 20px;
    border-radius: 10px;
    font-size: 14px;
    font-weight: 500;
  }
  .error { background-color: #fff5f5; color: #c53030; border: 1px solid #feb2b2; }
  .success { background-color: #f0fff4; color: #276749; border: 1px solid #9ae6b4; }
}

```

```

.input-group {
  margin-bottom: 20px;
}
.input-group label {
  display: block;
  margin-bottom: 8px;
  color: #4a5568;
  font-size: 14px;
  font-weight: 500;
}
.input-group input {
  width: 100%;
  padding: 14px;
  background-color: #f7fafc;
  border: 2px solid transparent;
  border-radius: 12px;
  font-size: 15px;
  transition: all 0.3s ease;
  box-sizing: border-box;
  color: #2d3748;
}
.input-group input:focus {
  border-color: #8ec5fc;
  background-color: #ffffff;
  outline: none;
  box-shadow: 0 0 0 4px rgba(142, 197, 252, 0.2);
}
.input-group input::placeholder {
  color: #a0aec0;
}

```

```

.submit-btn {
  width: 100%;
  padding: 15px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  border: none;
  border-radius: 12px;
  font-size: 16px;
  font-weight: 600;
  cursor: pointer;
  transition: transform 0.2s ease, box-shadow 0.2s ease;
  margin-top: 10px;
}
.submit-btn:hover {
  transform: translateY(-2px);
  box-shadow: 0 8px 20px rgba(118, 75, 162, 0.3);
}
.submit-btn:active {
  transform: translateY(0);
}
.signup-link {
  text-align: center;
  margin-top: 24px;
  font-size: 14px;
  color: #718096;
}
.signup-link a {
  color: #667eea;
  text-decoration: none;
  font-weight: 600;
  transition: color 0.2s ease;
}
.signup-link a:hover {
  color: #764ba2;
}
`</style>

<div className="form-card">
  <h2 className="form-title">Welcome Back</h2>
  <p className="form-subtitle">Please enter your details to sign in.</p>

  {message.text && (
    <div className={`message ${message.isError ? 'error' : 'success'} `}>
      {message.text}
    </div>
  )}

  <form onSubmit={handleSubmit}>
    <div className="input-group">
      <label>Email Address</label>
      <input
        type="email"
        name="email"
        placeholder="you@example.com"
        value={credentials.email}

```

```

        onChange={handleChange}
        required
      />
    </div>
    <div className="input-group">
      <label>Password</label>
      <input
        type="password"
        name="password"
        placeholder="Enter your password"
        value={credentials.password}
        onChange={handleChange}
        required
      />
    </div>
    <button type="submit" className="submit-btn">Sign In</button>
  </form>

  <p className="signup-link">
    Don't have an account? <Link to="/signup">Sign Up</Link>
  </p>
</div>
</div>
);
};

export default SignIn;

```

[Sign In](#)[Sign Up](#)

Welcome Back

Please enter your details to sign in.

Email Address

Password

Sign In

Don't have an account? [Sign Up](#)

Welcome page for the user After successful login

```
import React from 'react';
```

```
import { useNavigate } from 'react-router-dom';
```

```
const Welcome = () => {
```

```
  const navigate = useNavigate();
```

```
  const handleLogout = () => {
```

```
    // Later, you will add logic here to clear user sessions or tokens
```

```
    console.log("User logged out");
```

```
    navigate('/signin');
```

```
  };
```

```
  return (
```

```
    <div className="container">
```

```
      <style>{`
```

```
        .container {
```

```
          display: flex;
```

```
          justify-content: center;
```

```
          align-items: center;
```

```
          min-height: 100vh;
```

```
          background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
```

```
          font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
```

```
        }
```

```
        .welcome-card {
```

```
          background: white;
```

```
          padding: 50px 40px;
```

```
          border-radius: 12px;
```

```
          box-shadow: 0 8px 24px rgba(0,0,0,0.1);
```

```
          width: 100%;
```

```
    max-width: 450px;
    text-align: center;
}
.welcome-icon {
    font-size: 48px;
    margin-bottom: 16px;
}
.welcome-title {
    color: #333;
    margin-bottom: 12px;
    font-size: 28px;
    font-weight: 600;
}
.welcome-message {
    color: #666;
    margin-bottom: 30px;
    font-size: 16px;
    line-height: 1.5;
}
.logout-btn {
    padding: 12px 30px;
    background: #ff4757;
    color: white;
    border: none;
    border-radius: 8px;
    font-size: 16px;
    font-weight: 600;
    cursor: pointer;
    transition: background 0.3s ease, transform 0.1s ease;
}
```

```
.logout-btn:hover {
    background: #ff6b81;
}

.logout-btn:active {
    transform: scale(0.98);
}

`</style>
```

```
<div className="welcome-card">
  <div className="welcome-icon"><img alt="Welcome icon" data-bbox="100 100 150 150" /></div>
  <h1 className="welcome-title">Welcome, Parth Patil!</h1>
  <p className="welcome-message">
```

You have successfully logged into the system. Your authentication flow is working perfectly.

```

    </p>
    <button onClick={handleLogout} className="logout-btn">
      Log Out
    </button>
  </div>
</div>
);
};

```

```
export default Welcome;
```




Welcome, Parth Patil!

You have successfully logged into the system. Your authentication flow is working perfectly.

[Log Out](#)

This defines the schema for the user data as specified in the project requirements (First Name, Last Name, Email, and Password).

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  firstName: { type: String, required: true },
  lastName: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }
});

module.exports = mongoose.model('User', userSchema);
```

Initializes the Express server and connects it to MongoDB.

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const app = express();
const PORT = 5000;

app.use(cors());
app.use(express.json()); // Allows parsing of JSON data from frontend

const mongoURI = "mongodb://localhost:27017/mca_auth_db";
mongoose.connect(mongoURI)
  .then(() => console.log("MongoDB connected successfully"))
  .catch(err => console.log("MongoDB connection error:", err));

app.get('/', (req, res) => {
  res.send("Backend Server is Running");
});
```

```
});
const authRoutes = require('./routes/auth');
app.use('/api/auth', authRoutes);
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

File contains the logic for Sign Up and Sign In

```
const express = require('express');
const router = express.Router();
const User = require('../models/User');

// Sign Up Route
router.post('/signup', async (req, res) => {
  try {
    const { firstName, lastName, email, password } = req.body;

    // Requirement: Validation for required fields
    if (!firstName || !lastName || !email || !password) {
      return res.status(400).json({ message: "All fields are required" });
    }

    // Check if user already exists
    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(400).json({ message: "User already exists" });
    }

    const newUser = new User({ firstName, lastName, email, password });
    await newUser.save();
```

```
    res.status(201).json({ message: "User registered successfully" });  
  } catch (error) {  
    res.status(500).json({ message: "Server error", error: error.message });  
  }  
});
```

// Sign In Route

```
router.post('/signin', async (req, res) => {  
  try {  
    const { email, password } = req.body;  
  
    // Requirement: System verifies credentials  
    const user = await User.findOne({ email });  
  
    if (!user || user.password !== password) {  
      return res.status(400).json({ message: "Invalid email or password" });  
    }  
  
    // Requirement: Display success message  
    res.status(200).json({ message: "Login successful", user: { firstName: user.firstName }  
  });  
  } catch (error) {  
    res.status(500).json({ message: "Server error", error: error.message });  
  }  
});  
  
module.exports = router;
```